Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Bu ders üç öğrenci sunumundan oluşur. Sunumlar dersin mikroservis kitabındaki bölümlere dayanır ve kitabın FTGO (yemek siparişi ve teslimatı) örnek uygulamasını kullanır. Sunan kişiler aşağıda "bir öğrenci" diye anılır.

Genel Konular

- Birinci sunum: mikroservis mimarisine giriş
  - Mikroservis mimarisi, bir uygulamanın işlevsel parçalara bölünmesine dayanır. Bu parçalar (mikroservisler) birbirinden bağımsız geliştirilip dağıtılabilir (deploy edilebilir).
  - Monolitik mimaride bütün uygulama tek bir yapıdır. Her değişiklikte bütün sistem yeniden derlenip dağıtılır. Mikroserviste her hizmet bağımsızdır, bu da ekiplerin paralel ve hızlı çalışmasını sağlar.
  - Temel iki özellik: bağımsızlık ve esneklik.
    - Bağımsızlık: her mikroservis kendi işlevini ve kendi veri tabanını yönetir. Sipariş, bildirim ve ödeme servisleri örneğinde sipariş servisindeki bir güncelleme diğer ikisini etkilemez.
    - Esneklik: yalnız gereken servis ölçeklenebilir ya da güncellenebilir (ör. yalnız ödeme servisi). Her servisin konfigürasyonu ayrıdır ve her servis farklı bir teknoloji yığınıyla yazılabilir. Örnek: ödeme servisine Dynatrace ya da OpenTelemetry ile izleme (trace) eklemek performansı biraz düşürse de para akan yerde değerlidir ve diğer servislere yansımaz.
  - Avantajlar ve zorluklar (ekrandan, 00:10:00)
    - Avantajlar: hızlı geliştirme ve dağıtım; kolay bakım ve daha iyi ölçeklenebilirlik; ekipler arasında bağımsız çalışma imkânı.
    - Zorluklar: karmaşık yapının yönetimi; veri tutarlılığı sorunları (her servisin ayrı veri tabanı vardır); izleme ve hata ayıklama zorlukları.
  - 4+1 görünüm modeli (ekrandan, 00:11:00)
    - Mantıksal görünüm: yazılımcıların oluşturduğu sınıflar ve paketler arasındaki ilişkiler.
    - Uygulama (implementation) görünümü: derleme sisteminin ürettiği modüller (JAR dosyaları) ve bileşenler ile bağımlılıkları.
    - Süreç görünümü: çalışma zamanında bileşenlerin (süreçlerin) birbiriyle iletişimi.
    - Dağıtım görünümü: süreçlerin fiziksel ya da sanal makinelerle eşleştirilmesi.
    - Senaryolar (+1): dört görünümü bir araya getirip "canlandıran", aralarındaki iş birliğini açıklayan senaryolar.
  - Katmanlı mimari (ekrandan, 00:12:20)
    - Sunum katmanı (kullanıcı arayüzü, dış API'ler), iş mantığı katmanı (uygulamanın kuralları), veri katmanı (veri tabanı ile etkileşim, CRUD).
    - Avantaj: basit ve anlaşılır, geliştiriciler arasında iş bölümü kolay.
    - Dezavantaj: katmanlar arası sıkı bağımlılık; veri katmanındaki bir değişiklik bütün sistemi etkileyebilir; test zorlaşır.
  - Altıgen (hexagonal) mimari (ekrandan, 00:13:05)
    - İş mantığı merkezdedir ve adaptörlerle çevrilidir. Inbound adaptörler (ör. controller sınıfı, mesaj tüketicisi) dış talepleri iş mantığına iletir. Outbound adaptörler (ör. DAO, mesaj üreticisi) iş mantığından gelen talepleri dış sistemlere (veri tabanı, mesaj aracısı) iletir.

    * Avantaj: iş mantığı diğer bileşenlerden bağımsızdır; test kolaylaşır, modülerlik artar.
    * Dezavantaj: başlangıç tasarımı daha karmaşık, kurulum maliyeti daha yüksek.
    * Sunan öğrenciye göre günümüz mikroservislerinde genellikle altıgen mimari tercih edilir.
  - FTGO örneğiyle pratik kullanım (ekrandan, 00:14:10)
    * Sipariş (Order), restoran, mutfak (Kitchen), teslimat (Delivery), muhasebe (Accounting) ve bildirim (Notification) servisleri; her birinin kendi REST API'si ve kendine özel veri tabanı vardır.
    * API Gateway mobil uygulamalardan gelen istekleri ilgili servislere yönlendirir. Servisler arası iletişim REST API ya da asenkron mesajlaşma ile yapılır.
    * Yalnız sipariş servisi güncellendiğinde diğer servisler kesintiye uğramaz. Yoğun dönemde yalnız sipariş servisi ölçeklenebilir (e-ticaret örneği); video platformları içerik akışını ayrı servislerle yönetir.
  - Sonuç: bağımsız servisler hızlı geliştirmeyi kolaylaştırır, ama her servis için ayrı dağıtım hattı (pipeline) ve ayrı konfigürasyon yönetimi gerektiği için DevOps yükü artar. Bu yüzden teknik altyapı buna uygun hazırlanmalıdır.
- İkinci sunum: Saga ile işlem (transaction) yönetimi
  - Transaction yönetimi: bir işlemin birden fazla tabloyu ya da veri tabanını değiştirdiği durumda verinin tutarlı kalmasının sağlanması. Örnek: yemek siparişi verildiğinde cüzdandan para düşülmesi, sipariş kaydının oluşturulması, kurye konumunun izlenmesi.
  - Zor kısım rollback'tir: bir adım başarısız olursa önceki adımlarda yapılan değişiklikler nasıl geri alınır ki kullanıcı mağdur olmasın.
  - ACID (ekrandan, 00:30:00)
    * Atomicity (atomiklik): işlemler bir bütün olarak ya yapılır ya hiç yapılmaz. Rollback bu ilkenin temelidir.
    * Consistency (tutarlılık): veri tabanı işlem öncesi ve sonrası tutarlı bir durumda kalır.
    * Isolation (yalıtım): bir işlem diğer işlemlerden etkilenmez.
    * Durability (dayanıklılık): işlem başarıyla tamamlandığında sonuç kalıcı olarak saklanır.
  - Monolitik yapıda çözüm (ekrandan, 00:31:10)
    * Tek veri tabanı olduğu için transaction yönetimi basittir. Spring gibi framework'ler ACID'e uygun yapıyı kolayca kurar.
    * Spring'de `@Transactional` ile işaretlenen metot (örnekte `BankService.transferMoney`) içinde bir hata olursa, framework veri tabanına rollback gönderir ve önceki değişiklikler geri alınır.
  - Mikroserviste sorun: her servisin ayrı veri tabanı vardır; tek bir veri tabanında rollback yetmez.
  - Geleneksel çözüm: X/Open XA Two-Phase Commit (ekrandan, 00:33:30)
    * Bütün veri tabanları ve servisler üzerinde işlem senkronize edilir. Amaç, bütün katılımcıların işlemi ya tamamen commit etmesi ya da tamamen rollback etmesidir.
    * Önce bir hazırlık aşaması vardır: bütün servislerin ayakta ve commit'e hazır olup olmadığı denetlenir. Sonra commit ya da rollback emri verilir.
    * Dezavantajlar: senkrondur, bütün servislerin aynı anda ayakta olması gerekir; bir servis çalışmıyorsa işlem yapılamaz ve gecikme olur. MongoDB gibi NoSQL veri tabanları ve Kafka, RabbitMQ gibi modern mesaj aracıları bu protokolü desteklemez.
  - Modern çözüm: Saga tasarım deseni (ekrandan, 00:36:50)
    * İşlem, her serviste ayrı çalışan yerel (local) transaction'lara bölünür. Bir yerel transaction bitince servis bir mesaj yayınlar, bu mesaj sıradaki servisin yerel transaction'ını tetikler.
    * Her adım için önceden bir compensating transaction (geri alma işlemi) tanımlanır. Bir adım başarısız olursa önceki adımlar ters sırayla telafi edilir.
    * Asenkron mesajlaşma ile veriler zamanla tutarlı hale gelir. Bir servis dursa bile mesaj aracısı mesajı servis ayağa kalkana kadar tutar.
    * Yalnız veri tabanına yazan adımların geri alınması gerekir; yalnız okuma yapan adımların compensating transaction'ı yoktur.

  - $T_1 \dots T_n$ ve $C_n \dots C_1$ şeması (ekrandan, 00:39:00): $T_1 \dots T_n$ commit edilmişken T_{n+1} başarısız olursa $C_n, \dots, C_1$ sırayla çalışıp yapılan değişiklikleri geri alır.
  - FTGO "sipariş oluştur" sagası (ekrandan, 00:40:00 ve 00:40:45): altı adım; dördüncü adımda kart onayı başarısız olursa bilet `CREATE_REJECTED`, sipariş `REJECTED` durumuna geçer.
  - Saga iki türlü koordine edilebilir: orkestrasyon (bir merkezin adımları yönettiği) ve koreografi. Hangi adımların tamamlandığını tutmak için bir hafıza gerekir: bir SQL veri tabanı, mesaj aracısı ya da Redis gibi bellek içi bir yapı.
- Üçüncü sunum: mikroservis mimarisinin servislere ayrıştırılması (kitabın ikinci bölümü)
  - Servisleri belirlemek iteratif ve yaratıcılık isteyen bir süreçtir. Kurumdaki uzmanlarla konuşulup işin hikâyesi alınır, isimler servis ve sınıflara, fiiller operasyonlara dönüştürülür.
  - Üç adım: sistem operasyonlarını tanımlamak; servisleri tanımlamak (ör. sipariş servisi, restoran servisi); servis API'lerini ve servisler arası ilişkileri tanımlamak.
  - Sistem operasyonları (ekrandan, 00:52:30): komutlar (create, update, delete) ve sorgular (read). Doğru tanımlanmaları, bir serviste yapılan değişikliğin diğer servisleri en az etkilemesi için önemlidir.
  - Yüksek seviyeli alan modeli: sipariş verme hikâyesinden (ekrandan, 00:55:30) Consumer, Order, Restaurant, Courier, Location, PaymentInfo, DeliveryInfo, OrderLineItem, MenuItem, Address sınıfları ve ilişkileri çıkarılır (ekrandan, 00:58:30).
  - Ayrıştırma yöntemi 1: iş yeteneklerine (business capability) göre
    * İş yeteneği şirketin ne yaptığını tanımlar. İş yeteneği değişmez, nasıl yapıldığı değişir (banka çeki örneği: çek eskiden kâğıtla bankada, şimdi ATM ya da telefonla yatırılır).
    * FTGO yetenek hiyerarşisi ve servisler (ekrandan, 01:02:45): tedarikçi yönetimi iki servise (Courier, Restaurant), sipariş alma ve yerine getirme üç servise (Order, Kitchen, Delivery), muhasebe tek servise (Accounting) eşlenir.
    * Hangi yeteneğin hangi servise eşleneceği öznel bir karardır, iş süreçlerinin derinlemesine anlaşılmasına dayanır.
  - Ayrıştırma yöntemi 2: alt alan (subdomain) deseni (ekrandan, 01:07:50)
    * Domain-Driven Design'dan gelir. Tek bir alan modeli yerine her alt alanın kendi alan modeli vardır ve her alt alan bir servise eşlenir (Order, Delivery, Kitchen, Accounting).
    * Sınırlı bağlam (bounded context) servisleri birbirinden yalıtır. Bir kavram farklı ekiplerde farklı anlam taşıyabilir; her bağlam içinde ortak bir dil kurulur.
  - Ayrıştırma yönergeleri (ekrandan, 01:10:00): Single Responsibility Principle (SRP) ve Common Closure Principle (CCP).
  - Zorluklar (ekrandan, 01:15:30): ağ gecikmesi, veri tutarlılığı, senkron iletişim, dağıtık sistemin karmaşası ve tanrı sınıfları (God classes).
  - Servis API'lerinin tasarımı (ekrandan, 01:17:05): operasyonların servislere atanması, gerekli API'lerin belirlenmesi, senkron/asenkron iletişim seçimi, kendi kendine yeten servisler ve API Gateway kullanımı.

Hocanın Özellikle Vurguladığı Kısımlar

- Saga konusunun düzeyi
  - Hoca, Saga sunumundan sonra konunun girişe göre daha ileri bir konu olduğunu belirtti (00:43:14). Yani Saga, servislere ayrıştırma ve temel kavramlar oturduktan sonra anlamlı hale gelen bir desendir.
- Sunumlar kitabın bölümlerini izler
  - Hoca birinci sunumu giriş bölümünü özetleyen bir sunum olarak değerlendirdi (00:17:37). Dersin bu kısmı kitabın sırasıyla ilerler: giriş, servislere ayrıştırma, sonra Saga gibi ileri konular.
- Ders sonrası notlar
  - Üçüncü sunumda CAP teoremi "Consistency, Availability ve Partition" diye verildi (01:15:21). Kastedilen büyük olasılıkla partition tolerance'tır (ağ bölünmesine dayanıklılık). Teoreme göre ağ bölünmesi olduğunda tutarlılık ile erişilebilirlikten biri feda edilir; bu yüzden "üçünden en çok ikisi" denir.
  - Üçüncü sunumda CCP, "birbirine yakın işlemleri yapan servislerin bir sınıf içinde

toplanması ve birlikte güncellenmesi" diye anlatıldı (01:09:26). Common Closure Principle'ın bilinen ifadesi büyük olasılıkla şudur: aynı nedenle değişen sınıflar aynı pakette (mikroservis bağlamında aynı serviste) toplanmalıdır. Böylece bir değişiklik yalnız bir servisi etkiler.

Kısa Tekrar Notları

- Mikroservis: bağımsız geliştirilen ve dağıtılan, kendi veri tabanına sahip, tek sorumluluklu servisler.
- Monolit: tek yapı, her değişiklikte bütün sistemin yeniden dağıtımı.
- Avantaj: hızlı geliştirme, bağımsız ekipler, servis bazında ölçekleme ve teknoloji seçimi.
- Zorluk: karmaşıklık, veri tutarlılığı, izleme ve hata ayıklama, artan DevOps yükü.
- 4+1: mantıksal, uygulama, süreç, dağıtım görünümleri + senaryolar.
- Katmanlı mimari basit ama sıkı bağımlı; altıgen mimari iş mantığını adaptörlerle dış dünyadan yalıtır.
- ACID: atomiklik, tutarlılık, yalıtım, dayanıklılık.
- Monolitte `@Transactional` tek veri tabanında rollback ile yeter; mikroserviste yetmez.
- 2PC (XA): senkron, bütün katılımcılar ayakta olmalı; NoSQL ve Kafka/RabbitMQ desteklemez.
- Saga: yerel transaction'lar + mesajla tetikleme + ters sırada compensating transaction'lar; okuma adımlarının telafisi yok.
- Saga koordinasyonu: orkestrasyon ve koreografi.
- Ayrıştırma: önce sistem operasyonları (komut/sorgu), sonra servisler, sonra API'ler.
- İki ayrıştırma yöntemi: iş yeteneğine göre ve alt alana (DDD) göre.
- Yönergeler: SRP ve CCP. Engeller: gecikme, tutarlılık, senkron iletişim, karmaşa, tanrı sınıfları.
- API Gateway: farklı servislerin çıktılarını birleştirip tek noktadan sunar.

Detaylı Açıklamalar

- Bağımsızlık ve esneklik
  - Birinci sunum mikroservisin özünü iki kelimeyle özetler. Bağımsızlık, her servisin kendi verisini ve kodunu yönetmesidir; böylece bir servisin geliştirilmesi ve dağıtımı ötekileri beklemez. Esneklik ise bu bağımsızlığın getirdiği seçme özgürlüğüdür: yalnız yoğun servisi ölçeklemek, yalnız ödeme servisine ek izleme koymak ya da her servis için ayrı teknoloji seçmek mümkündür.
- Altıgen mimari neden mikroservise uyar
  - Katmanlı mimaride üst katman alt katmana doğrudan bağlıdır; veri katmanındaki bir değişiklik yukarıya yayılabilir. Altıgen mimaride iş mantığı yalnız port (arayüz) tanımlar; controller, mesaj tüketicisi, DAO ve mesaj üreticisi bu portları gerçekleyen adaptörlerdir. İş mantığı dış teknolojiyi bilmediği için tek başına test edilebilir ve adaptör değiştirmek iş mantığına dokunmaz.
- Monolitte transaction ve `@Transactional` (ekrandan, 00:31:10)
  - Slaytta görünen kod parçası:

```
@Service
public class BankService {

    @Autowired
    private AccountRepository accountRepository;

    @Transactional
    public void transferMoney(Long fromAccountId, Long toAccountId,
        Double amount) {
        // Gönderici hesabından para düşülüyor
        ...
    }
}
```

  - Metot içinden başka servis sınıflarına ve repository'lere gidilir. Örneğin hesap ya da kart doğrulaması başarısız olursa (kartın süresi geçmişse), o ana kadar yapılan

veri tabanı değişiklikleri `@Transactional` sayesinde geri alınır. Burada “servis” ayrı bir uygulama değil, aynı uygulamadaki bir sınıftır; bütün tablolar tek veri tabanında olduğu için tek bir rollback komutu yeterlidir.

- Two-Phase Commit’ten Saga’ya geçiş
  - 2PC doğru çalışır ama bütün katılımcıları aynı anda kilitleyen senkron bir protokoldür. Mikroservislerde servislerin bir kısmı geçici olarak kapalı olabilir; bu durumda işlem bekler. Ayrıca yaygın NoSQL veri tabanları ve mesaj aracıları XA’yı desteklemez. Saga bu yüzden kilitli tek bir global transaction yerine, her serviste kısa yerel transaction’lar ve bunları bağlayan mesajlar kullanır.
- FTGO “sipariş oluştur” sagası (ekrandan, 00:40:00, 00:40:45)
  - Başarılı senaryo:

```
1 Order Service      - Order'ı APPROVAL_PENDING durumunda oluştur
2 Consumer Service   - Tüketicinin sipariş verebildiğini doğrula
3 Kitchen Service    - Sipariş ayrıntılarını doğrula, Ticket'ı CREATE_PENDING durumund
4 Accounting Service - Tüketicinin kredi kartını onayla
5 Kitchen Service    - Ticket durumunu AWAITING_ACCEPTANCE yap
6 Order Service      - Order durumunu APPROVED yap
```

  - Dördüncü adımda kart onayı başarısız olursa beşinci adımda Ticket `CREATE_REJECTED`, altıncı adımda Order `REJECTED` durumuna geçer. İkinci adım yalnız okuma yaptığı için onun telafisi gerekmez.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.